

- Hybrid recommendation logic
 - Streamlit deployment for real-time recommendations
- The FOSS (free and open source) tools I used for this are Python, Pandas, Scikit-learn, Matplotlib and Streamlit.

This workflow is highly useful for students, developers, and open source administrators who want to understand how modern recommender systems are built from scratch.

Understanding recommendation systems

Broadly, recommendation systems fall into three categories.

Content-based recommendations: These suggest items similar to what the user liked before, using features like description, genre, or keywords.

Collaborative filtering: Here, recommendations are based on other users' behaviours and patterns, such as 'People who listened to this also listened to...'

Hybrid recommendations: These combine multiple strategies to improve both the accuracy and diversity of recommendations.

In this project, I will focus mainly on content + clustering hybrid recommendations, which work very well even without user history.

Dataset preparation

The first step is to collect and prepare the dataset. I worked with two publicly available datasets from Audible.

Audible_Catalog.csv : This contains basic information such as book name, author, genre, rating, number of reviews, price and listening time.

Audible_Catalog_Advanced_Features.csv: This contains richer metadata such as book description, narrator details, extended genre labels, and additional textual features.

Since both datasets share common columns, I merged them using 'book name' and 'author'. This produced a unified dataset that contains both structured numerical features and unstructured textual data.

This combination is essential because recommendation engines perform best when they have access to both quantitative engagement metrics and semantic meaning from descriptions.

Data cleaning and preprocessing

Real-world datasets are never clean. The Audible dataset had issues such as:

- Missing ratings
 - Incomplete genres
 - Duplicate book entries
 - Inconsistent formatting of listening time
- Using Pandas, I applied the following preprocessing steps.

Handling missing values: Numerical fields were filled with median values, and categorical fields were replaced with 'Unknown'.

Removing duplicates: Books with the same title and author were deduplicated to avoid repeated recommendations.

Standardising listening time: Listening time was converted into consistent minutes so it could be compared properly.

Genre normalisation: Genre labels were cleaned so that, for example, 'Self Help' and 'Self-Help' were treated the same way.

Data preprocessing is a crucial stage, because poor quality data produces poor recommendations.

Exploratory data analysis (EDA)

Before building models, the dataset should be explored to understand patterns. EDA helps answer questions like:

- Which genres dominate Audible's catalogue?
- Do highly rated books always have more reviews?
- Are expensive audiobooks rated better?
- Which authors consistently receive good ratings?

Genre distribution: Certain genres, like fiction, self-help, business, and mystery, appeared far more frequently. This indicates Audible's strong focus on mainstream listening categories.

Rating patterns: Most audiobooks were rated between 4.0 and 4.7, suggesting strong positivity bias in listener feedback.

Review count insights: Some books had thousands of reviews, while many had very few. This imbalance indicated the presence of popular blockbusters as well as lesser-known hidden gems.

Price vs rating: Interestingly, higher prices did not always mean better ratings. Many affordable audiobooks were equally well appreciated.

EDA gives valuable intuition and ensures that the recommendation engine is grounded in real data behaviour.

NLP feature extraction with TF-IDF

One of the most important parts of book recommendations is understanding the content of the book. Metadata alone (rating, price, genre) is not enough. Book descriptions carry semantic meaning, for example:

- 'A guide to productivity and habit formation...'
- 'A fantasy adventure set in a magical kingdom...'

To convert these descriptions into machine-readable form, I applied TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. This assigns weights to important words — words common in every description get a low weight while rare but meaningful words get a high weight. For example:

- 'the', 'and' → low importance
- 'neuroscience', 'dragon', 'entrepreneurship' → high importance

Thus, every book description is transformed into a numerical vector. These vectors form the foundation of content similarity.